Reflective Graph: Formal Model

Shahzada Saeed · MSc Computer Science

1. The question and its scope

Can a computational system represent and change its own rules, while retaining a precise account of what each change means? Reflective Graph investigates this question through graph rewriting. A state contains both application data and descriptions of the rules that transform it. Computation consists of admitted structural changes to this single graph.

Three distinctions guide the model. Representing rules internally does not remove the evaluator that interprets their descriptions. A stable structure may preserve useful information, but stability alone cannot establish correctness or meaning. Finally, an order derived from recorded dependencies describes the observed computation without necessarily explaining every possible alternative.

The formal results address these distinctions separately: a guarded rule-version discipline, conditions for particular forms of stability, a partial order over recorded events, and an encoding of weak-head lambda evaluation. Their assumptions are part of the results.

2. Atoms, fields and rule descriptions

2.1 The carrier

A state is a finite typed graph $G = (A, F, s, t, \tau, \alpha)$. Here A is the set of atom identities, F is the set of directed fields, s and t give their source and target, τ assigns kinds and record roles, and α contains the permitted attributes. Every field belongs to its source atom and occupies an ordered slot, zero or one. Kinds determine which slots exist and what they may reference.

Atom kind	Interpretation
Constructor	A labelled object, such as a data value or a syntax constructor.
PatternVariable	A placeholder constrained by its admitted kinds and binding mode.
RuleHead	The persistent identity of a rule.
Nat	A cell in a finite predecessor chain representing a natural number.
Record	An addressable relationship, membership or metadata item.

An `ObjectLink` record represents a labelled relationship between two constructors. Its fields identify the endpoints. Thus the relationship itself can be matched or replaced: it has an atom identity, just as its endpoints do. Field arrows express the internal record structure and are not themselves additional addressable atoms.

Fields and attributes of an existing atom are immutable. Rewriting changes a relationship by consuming its record and producing a fresh one. Each state is finite, although an unbounded execution may create arbitrarily many atoms or revisit an observation indefinitely.

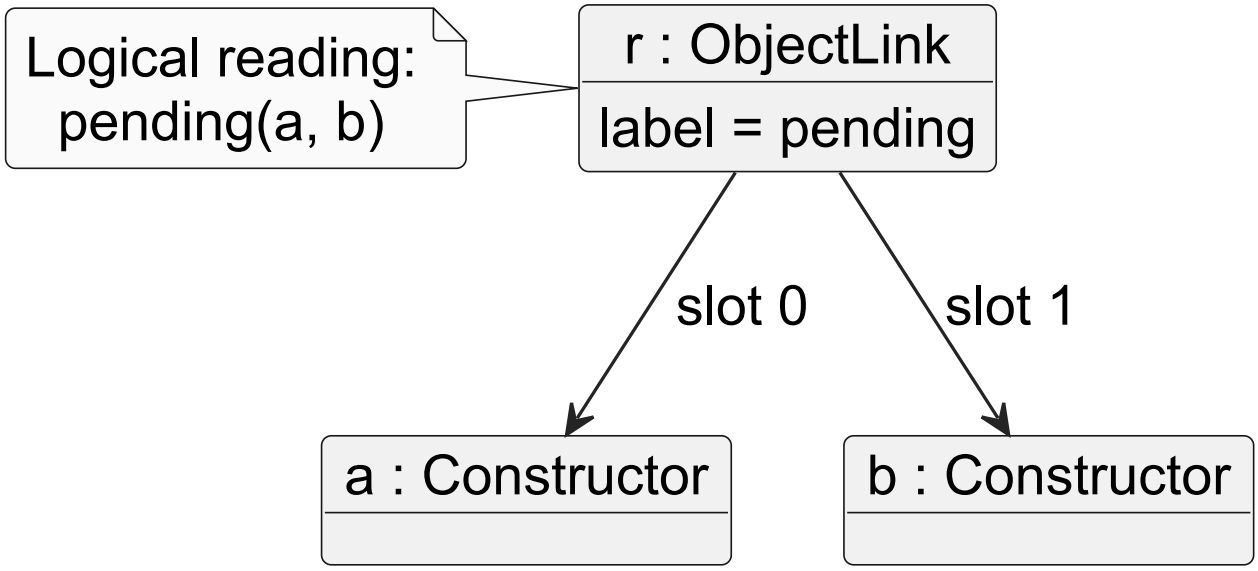


Figure 1. An addressable relationship is a record atom whose source-owned fields point to its endpoints.

2.2 Executable rules and quotation

Record roles distinguish `Lhs`, `Rhs`, `Rank`, `Active`, `ModeObject`, `ModeMeta` and `Revision`, as well as `ObjectLink`. Membership and rank records use their first field for the owning head and their second for the referenced item or number. Activation, mode and revision records refer to their owner.

Rule descriptions require quotation: a record describing a pattern must not immediately act as an application record. A membership record is operative when no membership record quotes it. Its second field identifies syntax owned by its live rule head. Well-formedness rejects rootless membership cycles and requires exclusive ownership of quoted syntax. An inert membership template therefore cannot independently change the visibility of another live atom.

An unquoted RuleHead is live. Every live head has one rank, mode and current revision, including when it is inactive. An Active record makes its description executable. Its item sets are

$$L(h) = \{x \mid \text{operative Lhs}(h, x)\}, \qquad R(h) = \{x \mid \text{operative Rhs}(h, x)\}.$$

The shared interface $K(h) = L(h) \cap R(h)$ identifies preserved items. Active descriptions are closed under concrete outgoing fields, and a replacement variable must already occur on the left. Rank chains are private, finite and acyclic.

Object-mode rules operate outside all live rule representations. Meta-mode rules can inspect the full carrier. A meta-rule may change a target's mode, thereby changing its visibility. These semantic modes are separate from user access permissions in the hosted application.

Literal new rule heads are permitted in a right-hand side only as complete, closed fresh packages. Their metadata and body are copied together; the installing rule's outer membership records are not copied. The new head becomes live while its own memberships continue to quote its syntax. Package variables are copied as the new rule's literal syntax rather than substituted as variables of the installer. This permits finite nested descriptions without an infinite tower of representation.

3. What constitutes a rewrite

3.1 Matching and context

A binding maps distinct pattern items injectively to compatible host atoms. It preserves required kinds, labels and field incidence. Repeated use of one variable refers to the same atom. A meta-variable marked `AppliedRule` is bound to the currently executing head, allowing a rule to refer specifically to itself.

For a binding σ, the consumed set is $C = \sigma(L \backslash K)$. The ground left context L_σ also includes the outgoing closure of the matched items. Its preserved part is $K_\sigma = L_\sigma \backslash C$. This completion matters when a matched record points to further structure that the pattern did not explicitly name. The added support is read and preserved, and must satisfy the source rule's visibility restriction.

The ground right context retains K_σ and adds fresh copies of concrete right-only items. Preserved atoms retain their identities, fields and attributes. Jointly created records may refer to one another where the instantiated description permits it.

3.2 Admission and atomicity

Deleting an atom removes its owned fields. A surviving atom may not retain a field to a consumed target; this is the dangling condition. The engine then constructs a candidate post-state and checks complete graph validity, revision lifecycle and the rank guard described below. A rewrite exists only when all these conditions hold. Failed admission leaves the input state unchanged.

For an injective ground match, the delete/preserve/create operation agrees with double-pushout rewriting on the expanded typed carrier, up to fresh identity renaming (Ehrig, Prange and Ehrig, 2006). Injectivity excludes identification conflicts. Closure includes outgoing fields of deleted atoms, and the dangling condition excludes incoming fields from surviving atoms. Removing the left-only part and adjoining the fresh right-only part therefore gives the corresponding pushout construction.

This correspondence concerns the instantiated carrier step. Substitution precedes it, while complete validation and reflective guards add application conditions. It does not equate the whole language with every form of attributed graph transformation.

4. Self-modification and bounded revision depth

4.1 Revisions distinguish executable versions

The representation $\mathrm{Rep}_G(h)$ contains a live head, its owned operative records, quoted syntax and expansions, and rank chain. Its body representation excludes only the current Revision record. Entry into the live set requires a fresh revision; exit consumes the current one. A continuing head changes revision exactly when its body representation changes. Syntax, rank, mode and activation changes therefore identify a new executable version. Revision-only churn is rejected.

If source rule p creates or substantively alters live rule q, admission requires

$$\mathrm{rank}_{G'}(q) < \mathrm{rank}_G(p).$$

The source rank is measured before the rewrite and the target rank afterwards. A rule can modify itself by lowering its rank. A higher-ranked supervisor can later restore a lower-ranked rule, provided the same inequality holds. Deleting or deactivating a rule must also satisfy the appropriate representation and revision conditions.

4.2 The version-depth result

Write $e \prec_v e'$ when event e installs or alters a rule q with fresh revision ρ, and a later event e' executes precisely that revision. Revision continuity ensures that q retains the rank installed by e until e': any intervening alteration or exit would consume ρ.

Proposition. An admitted execution has no infinite chain of $\prec_v$ links. From a finite initial graph whose largest live rank is M, every such chain has at most M links.

Argument. The rank guard and revision continuity give a strictly decreasing natural-number rank across each link. New or altered ranks are below an existing source rank, so induction also bounds every live rank by M. A strictly descending sequence beginning at most at M has at most M decreases.

The result bounds a chain of executable versions. Total rule-editing activity can still be infinite: a rank-two supervisor can repeatedly restore a lower-ranked rule, replacing its revision each time. Ordinary object computation can likewise continue indefinitely. This permits persistent adaptation but leaves convergence as a separate question.

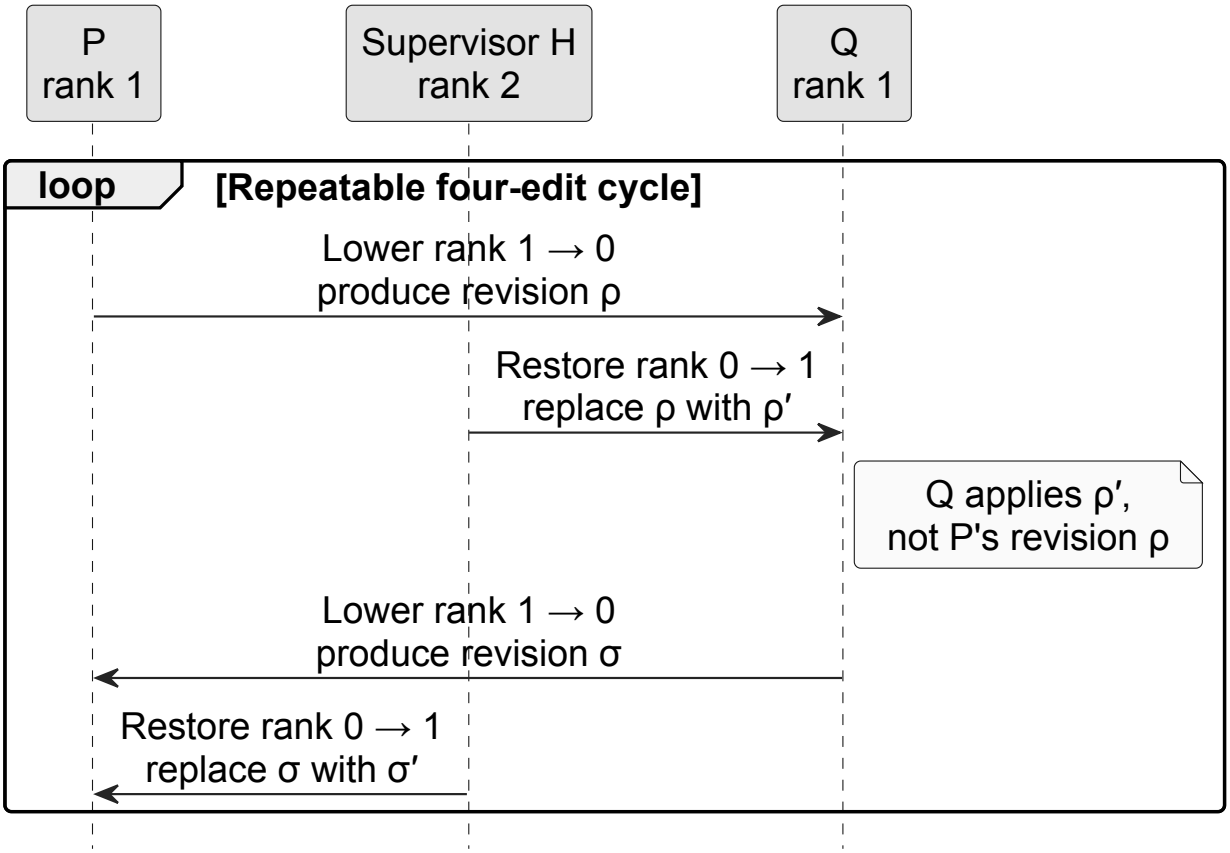


Figure 2. Supervised restoration permits repeated modification while each exact-version chain remains bounded.

A stronger edit bound is available for a finite, complete quotient of the reachable carrier states. Give an edge cost one when it changes a rule and zero otherwise. Total edit cost is bounded exactly when no reachable cycle has positive cost. Some growing systems admit a checked abstraction that discards inert, unobserved atoms. The abstraction must preserve the relevant transitions; an incomplete search cannot certify this condition.

5. Stability, recovery and continuing computation

5.1 The observation must be specified

An observer selects a field-closed part of the graph; equality may then be considered up to identity renaming. A normal form has no admitted rewrite. An observed fixed point preserves the selected information under the transitions being considered. These properties differ when internal computation continues outside the observation.

For example, a rule replacing `pending(x,y)` by `done(x,y)` while retaining an independent value record can finish without losing its payload. Rules alternating `pending` and `done` may preserve that same payload forever. A rule deleting every value can also reach a normal form. Consequently, a useful stability claim specifies both the observation and the information it must retain.

For the restricted program containing only the fixed pending-to-done processor and compatible data, each step consumes one pending record and creates none. Every remaining pending record has an admitted occurrence. Starting with k records therefore takes exactly k steps. Distinct replacements commute, giving local confluence; termination then yields a unique normal form by Newman's theorem (1942).

5.2 Recovery in a checked state space

Self-stabilisation concerns recovery to legitimate behaviour from a declared class of states (Dijkstra, 1974). For a finite complete reachable quotient, let L be a closed legitimate region. Require preservation of the chosen payload, a successor at every state outside L, and no cycle outside L. Define $V = 0$ on L and, outside it,

$$V(G) = 1 + \max_{G \to H} V(H).$$

Each outside transition decreases V, so every maximal execution enters L within its initial value. Stable outputs require those outputs also to be preserved within L. The conclusion covers the explored transition system, including admitted rule changes. Exhaustion of the exploration budget leaves the result unresolved.

5.3 Structural workflow contracts

A larger but restricted family avoids enumerating every reachable state. Each task has a persistent ticket, immutable payload and one status. A contract assigns natural weights to statuses, with terminal statuses of weight zero. Every permitted processing route from a to b, creating children with stages C, must satisfy

$$w(a) > w(b) + \sum_{c \in C} w(c).$$

The total task weight W decreases on processing. Permitted administrative rewrites preserve W and keep the processor family closed, including an available processor for every unfinished task. The recogniser checks these structural conditions conservatively.

Finite nonrenewable edit permits give a bound in terms of W and the remaining permits. Unlimited administration requires service assumptions. The implemented FIFO policy retains task identities across rule revisions and allows at most B administrative commits between processing commits. For a finite initial workload in the recognised class, completion takes at most $(B + 1)W_{\text{initial}}$ commits. Fairness is attached to persistent work, so replacing a rule does not reset a waiting task's position.

Different allowed routes can produce different terminal assignments. Arbitrary corruption, continuing arrivals and unrestricted changes to processing logic fall outside this completion theorem.

5.4 Local convergence under growth

Continuing arrivals motivate a different property: each established task may approach its own solution even while new tasks keep the whole graph changing. This is coordinatewise convergence, which need not make the maximum error over all current tasks vanish.

The numerical application studies retry and delegation probabilities under declared service, fresh-read and disturbance assumptions. Rust computes error bounds and publishes checked results through graph rewrites. The underlying sufficient arguments concern the specified numerical update system. They do not establish convergence for arbitrary self-modifying programs or prove that host arithmetic has become graph-resident computation.

6. Temporal order and explanations

An event records its source revision and binding, consumed set C, preserved context K, produced set P and read set R. Reads include the completed match and the source rule representation, including atoms also consumed.

For events $i < j$ in a recorded execution, add a dependency when

$$P_i \cap (R_j \cup C_j) \neq \varnothing \quad \text{or} \quad R_i \cap C_j \neq \varnothing.$$

Every edge follows the execution index, so the graph is acyclic and its transitive closure is a strict partial order. It describes resource dependence without requiring a physical clock, in the tradition of dependency-based temporal order (Lamport, 1978). Shared reads alone introduce no edge.

The order is partial evidence about applicability. Removing quotation may enable a rule without producing an atom in its later positive read footprint. Absence conditions and other alternative histories therefore require further analysis.

Selected provenance recursively follows recorded births and reads for a chosen observation. It identifies support in that history; it does not by itself establish that every supporting event was necessary. Bounded intervention analysis explores declared alternatives and can report minimal supports within that finite domain. These qualifications preserve the difference between an explanation of what happened and a proof of what had to happen.

7. Expressiveness and task-relative interpretation

7.1 Lambda evaluation

Finite lambda terms compile into constructor and object-link records together with a closure machine. The machine has control, environment and argument-stack components. Push records an argument, Beta enters a function body with an extended environment, and two lookup operations resolve variable indices. Because carrier matching is injective, eleven rules cover the sharing cases of these four schemes.

The representation supports closed terms and open terms supplied with an explicit free-variable context. Under its scoping and structural invariants, administrative steps preserve readback and Beta performs one capture-avoiding weak-head reduction. Administrative-only execution is finite: environment age and syntax height provide a decreasing measure. With unrestricted resources and no external interference, source and encoded weak-head termination therefore agree.

This establishes a uniform computational encoding beyond the individual examples. The strategy does not reduce underneath lambdas. Obsolete machine cells remain stored, and arbitrary cyclic source terms are outside the compiler’s input language.

7.2 What an interpretation establishes

A task supplies an observation O and a correctness relation between inputs and outputs. Agreement with that relation gives a usable interpretation of a graph computation. Stability of $O(G)$ alone does not supply the relation.

The adopted model applications examine population behaviour, finite learned descriptions, contextual distinctions and conservative action acquisition. Learning or fitting occurs in declared host components; compiled descriptions execute through resident rules. The action learner's safety conclusions rely on its stationary deterministic model class. Contradictory evidence can trigger repair or quarantine, while an unsupported prediction remains unknown. These results establish conditional task performance rather than intrinsic meaning.

A version-aware guest calculus additionally makes captured executable versions part of a small program's data. Supported return and delayed-return behaviours illustrate checked replacement. The outer interpreter and the graph admission rules remain fixed. The system thus contains executable representations of aspects of its semantics while retaining a specified external interpretation boundary.

8. Minimality and assurance

Minimality is assessed against declared encoding classes and cost measures. Reified relationships permit a conventional typed carrier and a clear ground rewriting correspondence. Alternative encodings can reduce the number of primitive forms while increasing record counts, description length or decoding work. Concrete snapshot measurements compare those costs for retained examples. They do not establish a globally smallest self-referential formalism.

The mathematical transition relation preserves validity because post-state validity is an admission premise. Establishing that Rust implements this premise correctly is a further obligation. The project combines explicit correspondence arguments, independent evaluators, property-based tests and selected Lean proofs. Those proofs concern stated abstractions rather than every implementation path.

The model consequently provides a controlled setting for executable reflection, conditional stability and inspectable temporal structure. Its broader significance is that these properties can be separated, implemented and tested within one carrier. A general theory of unrestricted self-modifying convergence, complete semantic self-interpretation and global minimality remains open.

References

Dijkstra, E.W. (1974) 'Self-stabilizing systems in spite of distributed control', *Communications of the ACM*, 17(11), pp. 643-644. Author manuscript.

Ehrig, H., Prange, U. and Ehrig, K. (2006) *Tutorial on Fundamentals of Algebraic Graph Transformation*. TU Berlin and University of Leicester. Tutorial.

Lamport, L. (1978) 'Time, clocks, and the ordering of events in a distributed system', *Communications of the ACM*, 21(7), pp. 558-565. Article.

Newman, M.H.A. (1942) 'On theories with a combinatorial definition of equivalence', *Annals of Mathematics*, 43(2), pp. 223-243. Article.